

testing, and releasing software. With containers, developers can create consistent environments that make testing and deploying new versions of apps faster and more reliable. Another huge advantage of containers is their portability. This means apps run the same way no matter where they are deployed—on different computers, servers, or in the cloud.

Because containers make collaboration easier, they've also boosted the adoption of DevOps, a practice that brings developers and operations teams together. Containers, combined with DevOps, have made software development faster, more scalable, and reliable.

In short, containers have completely transformed how we build, deploy, and manage apps. They're lightweight, quick to deploy, and can run anywhere. As technology continues to evolve, containers will remain a key part of the future of app development.

Containers vs virtual machines

While containers and virtual machines (VMs) both serve the purpose of isolating applications, it's important to understand the fundamental differences between them. Table 1 lists these differences. This distinction is crucial for developers and IT professionals looking to choose the right technology for their needs.

One of the most significant advantages of containers is their ability to create a consistent environment for applications, regardless of where they are deployed. They encapsulate the application along with all its dependencies, libraries, and settings into a single package. This ensures that the application behaves the same way, whether it's running on a developer's laptop, a testing server, or in a production cloud environment. This level of consistency and reliability reduces friction in the development lifecycle and enhances collaboration among team members.

Table 1: How virtual machines are different from containers

Feature	Virtual machines (VMs)	Containers
Isolation	Full OS isolation: Each VM runs its own OS, creating a barrier between them.	Application-level isolation: Containers share the host OS kernel while remaining separate.
Resource usage	Heavy: Requires more memory and CPU because each VM needs its own OS resources.	Lightweight: Utilises minimal resources by sharing the host OS, allowing for efficient resource allocation.
Speed	Slow to start: Booting a VM involves starting an entire OS, which can take time.	Containers can start almost instantly because they share the host OS kernel.
Portability	Limited: Moving VMs between environments often requires compatible hypervisors and can be complex.	Highly portable: Containers can run consistently across any environment that supports the container runtime, making them easy to deploy and migrate.

Kubernetes: Managing containers at scale

While Docker helps developers create and run individual containers, it becomes challenging to manage thousands of containers across multiple servers. That's where Kubernetes (K8s) comes in. Developed by Google and released as an open source project in 2014, Kubernetes helps manage, scale, and automate the deployment of containerised applications.

Think of Kubernetes like an air traffic controller for containers. It oversees how containers are deployed, ensuring they are working properly, scaling up or down depending on how much traffic there is, and monitoring their performance. For instance, during a big online sale, Kubernetes can add more containers to handle increased website traffic and remove them when traffic decreases to save resources.

In modern software development, apps are often built using a microservices architecture, which means that different functions of an app are split into smaller parts, each running in its own container. Kubernetes helps manage these containers, making sure they communicate and work together

smoothly, handling complex tasks like networking and load balancing. Here are some key features of Kubernetes.

Automated deployment and scaling: It automatically launches containers and adjusts their number based on real-time needs. For example, Amazon frequently experiences significant traffic spikes during major sales events. To prepare for these anticipated surges, Amazon utilises Kubernetes to automatically scale its services. Before the event, the company runs a baseline number of containers. As the sale approaches and traffic increases, Kubernetes intelligently adds more containers based on real-time demand, ensuring that the website remains responsive and capable of handling millions of requests per second.

Self-healing: If a container fails, Kubernetes automatically replaces it to keep the app running. For example, if a web application running in a container crashes due to an error, Kubernetes detects the failure and automatically creates a new instance of that container, ensuring that the application remains available and responsive to users.

Service discovery and load balancing: It assigns IP addresses